Lab 4 - Advanced Algorithms: Recursion, Backtracking, and GUI Programming

This lab introduces arrays, recursion, GUI programming, and advanced algorithmic thinking through six progressive tasks. Students will learn two-dimensional array manipulation, recursive method design, event-driven programming, and backtracking algorithms. The exercises progress from maze pathfinding through Game of Life simulation, GUI calculator development, game AI implementation, fractal generation, and culminate with an optional Sudoku solver, emphasizing problem decomposition, visual programming, and advanced data structure manipulation.

Contents

Memorize

Submission Deadline

Deadline to upload the solutions for all tasks is Saturday, 11:59 pm before the lab date.

General Information

The following tasks are to be worked on in fixed teams of two. Each team member must be able to explain all solutions. Please submit only one solution for each team of two. The submission must be a PDF file in our Moodle room with the name and matriculation number. Solutions must be in digital format with intermediate steps and detailed explanations (no handwritten scans). You can use any tool or drawing program of your choice to create the diagrams. If you have questions or need support, use the forum in our Moodle room and help each other.

1. Task 1: Maze Pathfinding with Recursion

Create a recursive pathfinding algorithm to help a mouse navigate through a labyrinth to reach cheese. This task introduces two-dimensional arrays, recursion, and backtracking algorithms.

A poor, hungry mouse sits in the upper left corner of a labyrinth (see sketch) and wants to reach a piece of cheese located in the lower right corner of the labyrinth. It can enter all non-hatched fields, but only via an edge shared by two adjacent fields. Help the mouse reach the cheese. Write a recursive method in Java that shows the mouse a path to the cheese.

> 🔥 **Tip**
>
> Your method must try for every possible field to find a path to the cheese via each of the four neighboring fields.

Your program should:

1. Represent the labyrinth using a 2D character array
2. Implement a recursive `findPath(int row, int col)` method
3. Use backtracking when paths lead to dead ends
4. Mark visited cells to prevent infinite loops
5. Display the final path through the maze

1.1. Preparation

Represent the labyrinth in a two-dimensional array. Use a method that checks all four directions and calls itself recursively to find the path. Tip: Use a marking character to mark the path that the mouse has taken, as the mouse should not go backwards. If you want, you can recreate the labyrinth with graphical methods and also enter the mouse's path to the cheese. A modification of the labyrinth or a larger number of fields is also possible.

1.2. Assistance

2D Array representation:

```java
public class MazeSolver {
    private char[][] maze = {
        {'#', '#', '#', '#', '#', '#', '#'},
        {'#', ' ', ' ', '#', ' ', ' ', '#'},
        {'#', ' ', '#', '#', ' ', '#', '#'},
        {'#', ' ', ' ', ' ', ' ', '#', '#'},
        {'#', '#', '#', ' ', '#', ' ', '#'},
        {'#', ' ', ' ', ' ', ' ', ' ', '#'},
        {'#', '#', '#', '#', '#', '#', '#'}
    };

    private int startRow = 1, startCol = 1;
    private int endRow = 5, endCol = 5;
}
```

2. Task 2: Conway's Game of Life Simulator

2.1. Preparation

Before implementing the solution, you must:

1. Design a class structure for the Game of Life simulator
2. Create a UML class diagram showing:
   - Class name (GameOfLife)
   - All private attributes (grid, rows, cols, generation counter, etc.)
   - All public methods with their parameters and return types (printGrid, nextGeneration, countNeighbors, etc.)
   - Initialization methods for different patterns (initGlider, initRandom, etc.)
   - Constructor(s)
3. Consider what data structures you need (2D boolean array for the grid)
4. Transfer your implementation into this class structure
5. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about the simulation architecture and state management before writing code.

2.2. Task

Create a simulation of Conway's Game of Life, a cellular automaton that demonstrates how complex patterns emerge from simple rules. This task introduces 2D array manipulation, simulation loops, and pattern evolution.

Conway's Game of Life is a zero-player game where cells on a grid live, die, or reproduce based on their neighbors:

- A live cell with 2-3 live neighbors survives
- A dead cell with exactly 3 live neighbors becomes alive
- All other cells die or stay dead

Your program should:

1. Create a 2D boolean array to represent the grid (true = alive, false = dead)
2. Initialize the grid with a pattern (e.g., glider, blinker, or random)
3. Implement `countNeighbors(int row, int col)` to count live neighbors
4. Create `nextGeneration()` to compute the next state
5. Display each generation and pause between updates
6. Run for a specified number of generations or until stable

2.3. Requirements

- Use a 2D array to represent the grid (minimum 20x20)
- Implement the four Game of Life rules correctly
- Display each generation in a readable format
- Handle edge cases at grid boundaries
- Allow user to choose initial pattern or use random seed

2.4. Assistance

Grid initialization and display:

```java
public class GameOfLife {
    private boolean[][] grid;
    private int rows, cols;

    public GameOfLife(int rows, int cols) {
        this.rows = rows;
        this.cols = cols;
        grid = new boolean[rows][cols];
    }

    public void printGrid() {
        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                System.out.print(grid[i][j] ? "\u2588 " : "\u00b7 ");
            }
            System.out.println();
        }
        System.out.println();
    }

    // Initialize with glider pattern
    public void initGlider(int startRow, int startCol) {
        grid[startRow][startCol + 1] = true;
        grid[startRow + 1][startCol + 2] = true;
        grid[startRow + 2][startCol] = true;
        grid[startRow + 2][startCol + 1] = true;
        grid[startRow + 2][startCol + 2] = true;
    }
}
```

Counting neighbors:

```java
private int countNeighbors(int row, int col) {
    int count = 0;
    for (int i = -1; i <= 1; i++) {
        for (int j = -1; j <= 1; j++) {
            if (i == 0 && j == 0) continue;
            int newRow = row + i;
            int newCol = col + j;
            if (newRow >= 0 && newRow < rows &&
                newCol >= 0 && newCol < cols &&
                grid[newRow][newCol]) {
                count++;
            }
```

```
13          }
14      }
15      return count;
16  }
```

3. Task 3: Simple GUI Calculator

3.1. Preparation

Before implementing the solution, you must:

1. Design a class structure for the GUI calculator
2. Create a UML class diagram showing:
   - Class name (Calculator) extending JFrame and implementing ActionListener
   - All private attributes (display field, button arrays, operands, current operation, etc.)
   - All public methods with their parameters and return types
   - The actionPerformed method (from ActionListener interface)
   - Helper methods for calculations (add, subtract, multiply, divide)
   - Constructor that sets up the GUI
3. Consider the GUI component hierarchy (which panels contain which components)
4. Plan your event handling strategy (how button clicks update state)
5. Transfer your implementation into this class structure
6. Only after completing the UML diagram should you begin coding

This preparation step ensures you think about GUI architecture, state management, and event-driven design before writing code.

3.2. Task

Create a graphical calculator application using Java Swing. This task introduces GUI programming, event handling, and building interactive applications.

Build a calculator with a graphical interface that performs basic arithmetic operations. This task helps you understand event-driven programming and user interface design.

Your program should:

1. Create a window with number buttons (0-9), operation buttons (+, -, *, /), equals, and clear
2. Display a text field showing the current input and result
3. Handle button clicks to build expressions
4. Evaluate expressions when equals is pressed
5. Clear the display when the clear button is pressed
6. Handle decimal numbers and basic error cases (division by zero)

3.3. Requirements

- Use Java Swing components (JFrame, JButton, JTextField, JPanel)
- Use GridLayout or BorderLayout for component arrangement
- Implement ActionListener for button events
- Display results in a readable format
- Handle basic error cases gracefully

3.4. Assistance

Basic GUI structure:

```java
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
```

```
4
5   public class Calculator extends JFrame implements ActionListener {
6       private JTextField display;
7       private JButton[] numberButtons;
8       private JButton[] operationButtons;
9       private double num1, num2, result;
10      private char operation;
11
12      public Calculator() {
13          setTitle("Calculator");
14          setSize(400, 500);
15          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
16          setLayout(new BorderLayout());
17
18          // Display field
19          display = new JTextField();
20          display.setEditable(false);
21          display.setFont(new Font("Arial", Font.BOLD, 24));
22          add(display, BorderLayout.NORTH);
23
24          // Button panel
25          JPanel buttonPanel = new JPanel();
26          buttonPanel.setLayout(new GridLayout(4, 4, 5, 5));
27
28          // Initialize buttons here
29          numberButtons = new JButton[10];
30          for (int i = 0; i < 10; i++) {
31              numberButtons[i] = new JButton(String.valueOf(i));
32              numberButtons[i].addActionListener(this);
33              numberButtons[i].setFont(new Font("Arial", Font.PLAIN, 20));
34          }
35
36          add(buttonPanel, BorderLayout.CENTER);
37          setVisible(true);
38      }
39
40      @Override
41      public void actionPerformed(ActionEvent e) {
42          // Handle button clicks
43          String command = e.getActionCommand();
44          // Implement logic here
45      }
46
```

```
47      public static void main(String[] args) {
48          new Calculator();
49      }
50  }
```

4. Task 4: Tic-Tac-Toe with AI Opponent

Create a Tic-Tac-Toe game with a computer opponent. This task combines 2D arrays, game logic, and basic AI using the minimax algorithm or simple heuristics.

Build a playable Tic-Tac-Toe game where a human player competes against the computer. The computer should make intelligent moves to challenge the player.

Your program should:

1. Use a 3x3 2D array to represent the game board
2. Display the board after each move
3. Allow the human player to choose positions
4. Implement computer AI that makes strategic moves
5. Check for win conditions (rows, columns, diagonals)
6. Detect draw conditions when the board is full
7. Allow replay after game ends

4.1. Requirements

- Use a 2D character array for the board
- Validate user input (position must be empty and valid)
- Implement win detection for all 8 possible winning lines
- Computer should block player wins when possible
- Display clear game status (whose turn, winner, draw)

4.2. Assistance

Game board structure:

```java
public class TicTacToe {
    private char[][] board;
    private static final char EMPTY = ' ';
    private static final char PLAYER = 'X';
    private static final char COMPUTER = 'O';

    public TicTacToe() {
        board = new char[3][3];
        for (int i = 0; i < 3; i++) {
            for (int j = 0; j < 3; j++) {
                board[i][j] = EMPTY;
            }
        }
    }

    public void printBoard() {
        System.out.println("  0 1 2");
        for (int i = 0; i < 3; i++) {
            System.out.print(i + " ");
            for (int j = 0; j < 3; j++) {
```

```
21                System.out.print(board[i][j]);
22                if (j < 2) System.out.print("|");
23            }
24            System.out.println();
25            if (i < 2) System.out.println("  -----");
26        }
27        System.out.println();
28    }
29
30    public boolean checkWin(char player) {
31        // Check rows
32        for (int i = 0; i < 3; i++) {
33            if (board[i][0] == player && board[i][1] == player &&
34                board[i][2] == player) return true;
35        }
36
37        // Check columns
38        for (int j = 0; j < 3; j++) {
39            if (board[0][j] == player && board[1][j] == player &&
40                board[2][j] == player) return true;
41        }
42
43        // Check diagonals
44        if (board[0][0] == player && board[1][1] == player &&
45            board[2][2] == player) return true;
46        if (board[0][2] == player && board[1][1] == player &&
47            board[2][0] == player) return true;
48
49        return false;
50    }
51
52    // Simple AI: Find winning move, block opponent, or choose random
53    public void computerMove() {
54        // Try to win
55        for (int i = 0; i < 3; i++) {
56            for (int j = 0; j < 3; j++) {
57                if (board[i][j] == EMPTY) {
58                    board[i][j] = COMPUTER;
59                    if (checkWin(COMPUTER)) return;
60                    board[i][j] = EMPTY;
61                }
62            }
63        }
```

```
64
65          // Try to block player
66          for (int i = 0; i < 3; i++) {
67              for (int j = 0; j < 3; j++) {
68                  if (board[i][j] == EMPTY) {
69                      board[i][j] = PLAYER;
70                      if (checkWin(PLAYER)) {
71                          board[i][j] = COMPUTER;
72                          return;
73                      }
74                      board[i][j] = EMPTY;
75                  }
76              }
77          }
78
79          // Choose center if available
80          if (board[1][1] == EMPTY) {
81              board[1][1] = COMPUTER;
82              return;
83          }
84
85          // Choose random available position
86          // Implementation here
87      }
88  }
```

5. Task 5: Recursive Fractal Tree Generator

Create a program that draws fractal trees using recursion. This task demonstrates recursive drawing, coordinate geometry, and visual pattern generation.

Implement a fractal tree generator that uses recursion to create beautiful branching patterns. Each branch spawns two smaller branches at angles, creating a tree-like structure.

Your program should:

1. Use Java graphics (JPanel, Graphics2D) to draw lines
2. Implement a recursive `drawBranch()` method
3. Start with a trunk and recursively draw smaller branches
4. Decrease branch length by a factor (e.g., 0.7) at each level
5. Split each branch into two branches at angles (e.g., ±30 degrees)
6. Stop recursion when branches become too small (base case)
7. Allow user to adjust parameters (angle, length factor, depth)

5.1. Requirements

- Use recursive method for drawing branches
- Apply trigonometry to calculate branch endpoints
- Implement proper base case to stop recursion
- Create a visually appealing tree structure
- Allow customization of tree parameters

5.2. Assistance

Basic fractal tree structure:

```java
import javax.swing.*;
import java.awt.*;

public class FractalTree extends JPanel {
    private int maxDepth = 10;

    @Override
    protected void paintComponent(Graphics g) {
        super.paintComponent(g);
        Graphics2D g2d = (Graphics2D) g;
        g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
                             RenderingHints.VALUE_ANTIALIAS_ON);

        // Start drawing from bottom center
        int startX = getWidth() / 2;
        int startY = getHeight() - 50;
        int length = 100;

        drawBranch(g2d, startX, startY, length, -90, maxDepth);
    }
```

```
21
22      private void drawBranch(Graphics2D g2d, int x1, int y1,
23                              double length, double angle, int depth) {
24          if (depth == 0 || length < 2) return;
25
26          // Calculate end point of current branch
27          int x2 = x1 + (int)(length * Math.cos(Math.toRadians(angle)));
28          int y2 = y1 + (int)(length * Math.sin(Math.toRadians(angle)));
29
30          // Draw the branch
31          g2d.setColor(new Color(139, 69, 19)); // Brown
32          g2d.drawLine(x1, y1, x2, y2);
33
34          // Recursively draw two smaller branches
35          double newLength = length * 0.7;
36          drawBranch(g2d, x2, y2, newLength, angle - 25, depth - 1);
37          drawBranch(g2d, x2, y2, newLength, angle + 25, depth - 1);
38      }
39
40      public static void main(String[] args) {
41          JFrame frame = new JFrame("Fractal Tree");
42          FractalTree tree = new FractalTree();
43          frame.add(tree);
44          frame.setSize(800, 600);
45          frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46          frame.setVisible(true);
47      }
48  }
```

Extending with user controls:

```
1  // Add sliders or input fields to control:                    Java
2  // - maxDepth (recursion depth)
3  // - angle spread (branch angle)
4  // - lengthFactor (how much branches shrink)
5  // - initial trunk length
```

6. Task 6: Sudoku Solver with Backtracking (Optional)

Create a Sudoku solver using recursive backtracking. This task builds on the maze pathfinding concepts and applies them to constraint satisfaction problems.

Implement a program that can solve 9x9 Sudoku puzzles using recursive backtracking:

- Read a partially filled Sudoku grid (use 0 for empty cells)
- Find empty cells and try numbers 1-9
- Use recursion to explore all possible solutions
- Backtrack when constraints are violated
- Display the solved puzzle

Your program should:

1. Represent the Sudoku grid as a 2D integer array
2. Implement `isValid(int[][] grid, int row, int col, int num)` to check constraints
3. Create a recursive `solveSudoku(int[][] grid)` method
4. Handle backtracking when no valid numbers can be placed
5. Display the completed puzzle

6.1. Requirements

- Check row, column, and 3x3 box constraints
- Use recursive backtracking algorithm
- Handle cases with no solution
- Display the grid in a readable format

6.2. Assistance

Sudoku grid representation:

```java
public class SudokuSolver {
    private static final int SIZE = 9;
    private static final int EMPTY = 0;

    private int[][] grid = {
        {5, 3, 0, 0, 7, 0, 0, 0, 0},
        {6, 0, 0, 1, 9, 5, 0, 0, 0},
        {0, 9, 8, 0, 0, 0, 0, 6, 0},
        {8, 0, 0, 0, 6, 0, 0, 0, 3},
        {4, 0, 0, 8, 0, 3, 0, 0, 1},
        {7, 0, 0, 0, 2, 0, 0, 0, 6},
        {0, 6, 0, 0, 0, 0, 2, 8, 0},
        {0, 0, 0, 4, 1, 9, 0, 0, 5},
        {0, 0, 0, 0, 8, 0, 0, 7, 9}
    };
}
```

Constraint checking:

```java
private boolean isValid(int[][] grid, int row, int col, int num) {
```

```
2       // Check row
3       for (int x = 0; x < SIZE; x++) {
4           if (grid[row][x] == num) return false;
5       }
6
7       // Check column
8       for (int x = 0; x < SIZE; x++) {
9           if (grid[x][col] == num) return false;
10      }
11
12      // Check 3x3 box
13      int startRow = row - row % 3;
14      int startCol = col - col % 3;
15      for (int i = 0; i < 3; i++) {
16          for (int j = 0; j < 3; j++) {
17              if (grid[i + startRow][j + startCol] == num) return false;
18          }
19      }
20      return true;
21  }
```

7. Lab Execution

If your program is not yet working without issue, we will try to correct this during the course of the lab. With good preparation, this should not be a problem. Every student is required to be able to explain their thought process at the beginning of the lab. By the end of the lab, the task needs to be completed. Of course, we will support you, but your personal commitment must also be clearly recognizable! Julian Moldenhauer, Furkan Yildirim, and Emily Antosch wish you lots of fun and success!